

TRACE: A Self-Evolving Skill Bank for Consistent, Limit-Aware LLM Agents

A Technical Report on the CAR-bench Challenge

Wenhao Wu^{* 1,2} Menghao Zhang^{* 2,3} Xin Wang^{* 2,4} Zhi Wang¹ Kun Shao² Jian Luan²

¹Nanjing University ²Xiaomi Inc.

³Beijing University of Posts and Telecommunications ⁴Tsinghua University

wenhaowu@smail.nju.edu.cn zhangmenghao1@xiaomi.com x-wang24@mails.tsinghua.edu.cn
shaokun@xiaomi.com zhiwang@nju.edu.cn

Abstract

Reliable deployment of LLM agents in user-facing products depends not on raw task-solving ability but on *consistency* and *limit-awareness*: behaving the same way across repeated trials, and recognizing when a request cannot, or cannot yet, be safely fulfilled. CAR-bench exposes this reliability gap in the domain of in-car assistants: an LLM-simulated user issues incomplete or ambiguous requests, requiring the agent to resolve uncertainty through multi-turn dialogue and tool use while strictly adhering to domain policies. Even frontier models show a substantial gap between what they can solve at least once (Pass@ k) and what they solve consistently across trials (Pass k). We bridge this gap with **TRACE** (TRAjectory-Contrastive Evolution), which iteratively improves a skill-based agent’s behavioral knowledge without modifying model weights. This knowledge is organized as a **Skill Bank** of modular, retrievable skills, each encoding a self-contained set of tool-use rules and behavioral guidelines. TRACE evolves this bank through an agentic self-evolution loop: after each evaluation round, it groups trajectories by the skills invoked and refines each skill by contrasting successful and failed behaviors. The updated bank then guides the next round, while the agent retrieves relevant skills at each inference turn. On GPT-5.5, TRACE improves consistency (Pass 3) by **34.6 points**, from 59.9% to 94.5%, while achieving a potential performance (Pass@3) of 98.5%. This shrinks the gap between potential and reliable performance to just **4.0 points**. Reliability also remains stable as the evaluation becomes stricter: Pass k drops by only **3.3 points** from $k=1$ to $k=3$. These results show that TRACE converts high model potential into stable, consistent performance gain.

1 Introduction

Deploying LLM agents in user-facing products demands more than raw task-solving capability [Jaech *et al.*, 2024; Guo *et al.*, 2025; Han *et al.*, 2025]. Modern agents combine chain-of-thought reasoning [Wei *et al.*, 2022; Wang *et al.*, 2023; Zhan *et al.*, 2026] with tool use and action [Yao *et al.*, 2023], enabling recent reasoning-centric models [Yan *et al.*, 2025; Zhang *et al.*, 2026] to tackle demanding agentic benchmarks [Hu *et al.*, 2025; Wu *et al.*, 2026], such as real-world software issue resolution [Jimenez *et al.*, 2024]. However, benchmark success does not guarantee reliable behavior in multi-turn interactions, where users may issue incomplete, ambiguous, or unsatisfiable requests and agents must manage the resulting uncertainty while adhering to domain-specific policies [Barres *et al.*, 2025]. Two properties are therefore essential: (i) *consistency*, producing stable behavior across repeated trials, and (ii) *limit-awareness*, recognizing when a request cannot, or cannot yet, be safely fulfilled instead of claiming unsupported success. Both properties are safety-critical for in-car assistants, where premature or unsupported actions can distract or endanger drivers.

Current LLMs are poorly aligned with both. **Consistency** remains fragile because, although models possess the meta-reasoning ability to recognize ambiguous requests and determine when clarification is needed, they do not reliably activate this ability across repeated trials [Hu *et al.*, 2026]. **Limit-awareness** is weakened by training objectives that favor plausible task completion over honest uncertainty, leading models to claim they can fulfill a request even when they lack the required capability rather than admit the limitation [Kalai *et al.*, 2025]. Together, these weaknesses create a *completion-compliance tension*: agents often prioritize satisfying the request over following policies, seeking clarification, or acknowledging unavailable capabilities. Because such failures are intermittent, the same agent may succeed in one trial but fail in another, exposing a gap between what it *can* do and what it does *reliably*.

We address this gap with **TRACE** (TRAjectory-Contrastive Evolution), a general, model-agnostic method that turns existing model capabilities into more consistent, limit-aware agent behavior. Rather than modifying model weights, TRACE improves the behavioral scaffolding around the model, in line with recent work on self-evolving agent harnesses [Chen *et al.*, 2026]. Unlike single-episode

* Equal contribution.

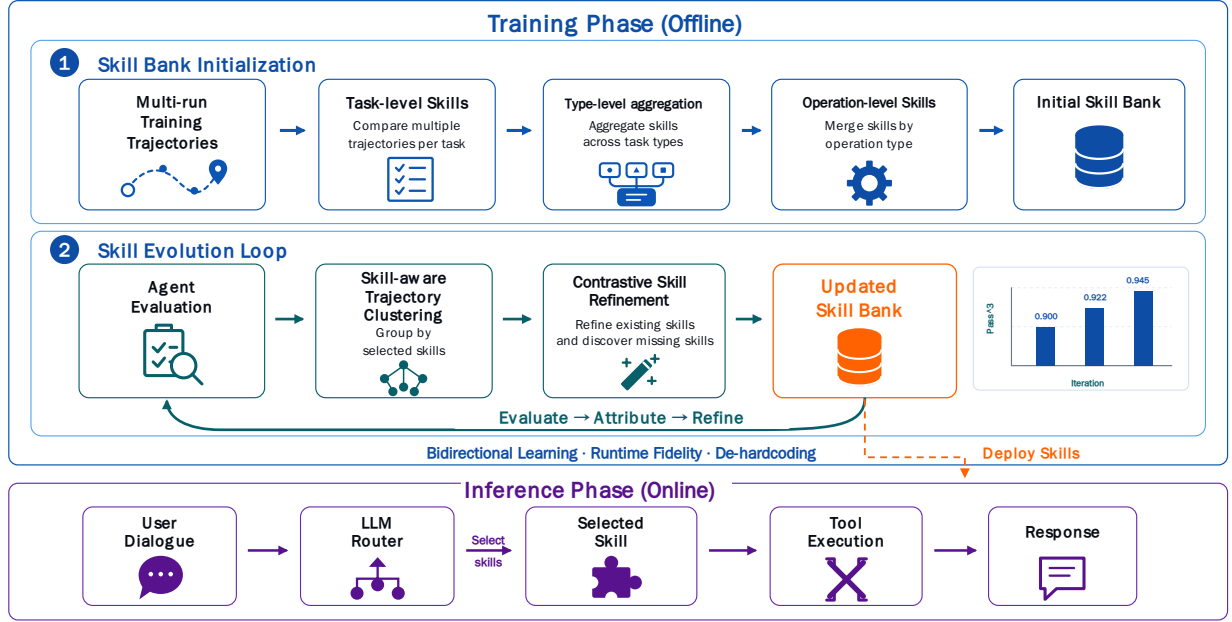


Figure 1: Overview of TRACE. **(Training)** The Curator initializes the Skill Bank bottom-up, then runs the evolution loop to iteratively refine it from the Actor’s evaluation trajectories. **(Inference)** At every turn, the deployed Actor re-selects the relevant skills for the current request and executes tools accordingly.

reflection methods [Shinn *et al.*, 2023], TRACE distills reusable behavioral knowledge *across* evaluation rounds into a persistent **Skill Bank** of modular, retrievable skills. Each skill encodes tool-use rules and behavioral guidelines, and the agent retrieves relevant skills at each inference turn.

TRACE evolves the Skill Bank through an LLM-driven closed loop: after each evaluation round, it groups trajectories by the skills invoked and refines each skill by contrasting successful and failed behaviors. This contrast reveals decisions that cause unreliable outcomes, such as acting on incomplete information, overlooking policy constraints, or claiming unavailable capabilities, and converts them into reusable guidance for *limit-awareness* and policy adherence. Two design choices keep the evolved skills general and transferable: i) operation-level organization abstracts skills from individual tasks, allowing knowledge learned in one scenario to transfer to others that require the same operation; and ii) a strict de-hardcoding constraint prevents skill updates from encoding memorized answers or environment-specific values, preserving only transferable tool-use and decision principles.

We summarize our contributions as follows.

- We introduce TRACE, a model-agnostic framework that self-evolves a modular Skill Bank from an agent’s evaluation trajectories without modifying model weights.
- We develop an agentic loop that refines skills by contrastive trajectory analysis, with operation-level organization and de-hardcoding to preserve transferability.
- We validate the evolved Skill Bank across multiple LLMs, showing substantial improvements in $\hat{\text{Pass}}^3$ and limited degradation in $\hat{\text{Pass}}^k$ as k increases, with gains that transfer across underlying models.

2 TRACE: A Self-Evolving Skill Bank

TRACE is organized around two agents, as shown in Figure 1. The **Actor** is the task-executing agent that converses with the user, calls tools and selects relevant skills to solve tasks. The **Curator** is the skill-optimizing agent that reads the Actor’s evaluation trajectories and rewrites the behavioral knowledge the Actor will act on next. The behavioral knowledge lives in a *Skill Bank* $\mathcal{B} = \{s_1, \dots, s_N\}$ of N skills: a set of modular, retrievable competencies stored as markdown `SKILL.md` files. Each skill is a pair $s_i = (d_i, b_i)$, where the one-line description d_i serves as its retrieval entry and the body b_i is a self-contained bundle of tool-usage rules and behavioral guidelines. At every turn the Actor selects a relevant subset $S \subseteq \mathcal{B}$ to augment its generation, while the Curator’s job is to *self-evolve* the bank from evaluation evidence, mapping $\mathcal{B}$ to an improved $\mathcal{B}'$ so that in later runs the Actor triggers and executes the right competency more consistently.

2.1 Initializing the Skill Bank

Before the optimization loop, the Curator bootstraps an initial bank $\mathcal{B}^{(0)}$ offline from tens of rounds of the Actor’s evaluation on the training set. The initialization follows a bottom-up process with two stages: hierarchical skill abstraction and skill decomposition. The first stage progressively abstracts task-specific experience into reusable operation-level knowledge, while the second adjusts the granularity of that knowledge for precise retrieval. Formally, the process is $\mathcal{B}^{\text{task}} \rightarrow \mathcal{B}^{\text{type}} \rightarrow \mathcal{B}^{\text{op}} \rightarrow \mathcal{B}^{(0)}$.

Hierarchical Skill Abstraction. The Curator consolidates the trajectories through three successive passes:

- **Task-level distillation.** For each task, the Curator compares its trajectories across evaluation rounds and distills a task-level skill that captures both successful behaviors and common failure patterns. This produces $\mathcal{B}^{\text{task}}$.
- **Type-level aggregation.** The Curator groups tasks by type and merges their task-level skills, removing task-specific details so that only behavior shared within each type remains. This produces $\mathcal{B}^{\text{type}}$.
- **Operation-level abstraction.** The Curator further merges skills across task types according to their underlying operations. Behaviors associated with the same operation are unified into a reusable competency, even when they arise from tasks with different surface goals. This produces $\mathcal{B}^{\text{op}}$.

Skill Decomposition. The Curator then decomposes broad skills in $\mathcal{B}^{\text{op}}$ into finer-grained skills, each covering a single, focused competency. This allows the Actor to retrieve precisely the knowledge required at each turn rather than broad or overlapping guidance, yielding the initial Skill Bank $\mathcal{B}^{(0)}$.

Hierarchical abstraction enables knowledge learned in one scenario to transfer to others that require the same operation, while decomposition keeps each skill sufficiently focused for precise and reliable retrieval.

2.2 Trajectory-Contrastive Skill Evolution

Let $\mathcal{T}^{(r)}$ be the set of trajectories the Actor produces when evaluated with bank $\mathcal{B}^{(r)}$. After each round, the Curator runs one pass of the loop, an update $\mathcal{B}^{(r+1)} = \Phi(\mathcal{B}^{(r)}, \mathcal{T}^{(r)})$: it clusters the trajectories in $\mathcal{T}^{(r)}$ by the skill each turn invoked, reconstructs the exact trajectory the Actor actually took and received at runtime, and rewrites each skill by contrasting the Actor’s successful and failed behavior.

Skill-aware Grouping. The Curator first partitions $\mathcal{T}^{(r)}$ by the skills each trajectory selected, sending trajectories that hit an existing skill s_i to that skill’s group $\mathcal{T}_i$ and setting aside those with an unrecognized name or no skill at all. This turns a stream of noisy trajectories into per-skill samples that can be attributed to a specific competency, each carrying the reward, tool calls, expected actions, and errors needed for that attribution.

Runtime-faithful Reconstruction. The Curator then renders each skill’s trajectories into structured text, with one key design decision: it *distinguishes what the Actor could actually see at runtime from what is only available offline*, and labels each piece accordingly. The shared prompt and base tools are shown once, while each task adds only its **capability changes** relative to that baseline. This separation lets the Curator use privileged information to diagnose *what* went wrong in a trajectory, while still judging the Actor’s decisions by exactly the affordances it faced, so that a skill is never rewritten to depend on knowledge the Actor will not have at runtime.

Contrastive Skill Refinement. Finally, the Curator rewrites in two modes. When optimizing an existing skill, it reads that skill’s paired success *and* failure trajectories and edits it directly, so it learns not only why the competency worked but also how it went wrong; if a skill has grown too large and complex, it is further split into finer-grained skills.

When mining for missing skills, the Curator inspects the set-aside no-skill trajectories and creates or revises a skill if a reusable, uncovered pattern recurs.

2.3 Per-Turn Skill Retrieval

While the optimization loop shapes *what* the skill bank contains, we now describe how the Actor puts it to use at inference time. The Actor never holds the full bank in context. Instead, only the descriptions $\{d_i\}$ stay resident in the Actor’s prompt, forming a compact index of the competencies available to it. Because each d_i is written to cover its skill’s triggering phrasings without overlapping the others, this index alone lets the Actor decide, at each turn, which competencies the current dialogue state calls for.

Retrieval by Self-Selection. Let h_t denote the dialogue history up to turn t . As N is small, the Actor selects the relevant skills by reading the resident descriptions against h_t , i.e. $S_t = \sigma(h_t, \{d_i\}) \subseteq \mathcal{B}$. This keeps the mechanism simple, and lets the same judgment that drives the Actor’s tool calls also decide which behavioral knowledge it pulls in.

Injection into Context. Once S_t is selected, the corresponding bodies $\{b_i : s_i \in S_t\}$ —the tool-usage rules, behavioral guidelines, mistakes to avoid, procedures, and so on—are appended to the Actor’s prompt for that turn. The Actor then samples its next action $a_t \sim \pi(\cdot \mid h_t, \{b_i : s_i \in S_t\})$, so that the retrieved competencies directly govern the tools it calls and how it handles the request.

Per-turn Refresh. Skill selection is recomputed each turn: $S_{t+1} = \sigma(h_{t+1}, \{d_i\})$, independent of S_t . The bodies injected on one turn are not carried over; the next turn starts again from the resident descriptions and re-selects against the updated dialogue state. This refresh keeps the context lean and, more importantly, lets the active competencies track the conversation as it evolves: a turn that begins as a clear request but proves underspecified, or one that hits a missing capability, pulls in exactly the skills that now apply rather than staying anchored to what was relevant earlier.

3 Experiments

Benchmark. We evaluate on CAR-bench [Kirmayr *et al.*, 2026], which extends agent testing to a safety-critical in-car assistant setting. An LLM-simulated user is given a persona and a hidden task instruction and exchanges text messages with the agent, which has native tool-calling access to 58 interconnected tools and must obey 19 domain policies. Tasks span three types: (i) *Base* tasks, which test ordinary task completion; (ii) *Hallucination* tasks, in which a required tool, parameter, or result is removed, so the agent must acknowledge the missing capability rather than fabricate one; and (iii) *Disambiguation* tasks, which introduce controlled ambiguity that the agent should resolve internally where possible and escalate to the user only when necessary.

Metrics. To measure consistency directly rather than incidentally, each task is run k times and aggregated over T tasks into two metrics: $\text{Pass}^{\wedge}k$ (solved in *all* k trials) captures reliability, while $\text{Pass}@k$ (solved in *at least one*) captures potential. The gap between the two isolates the *latent competence* a model possesses but cannot apply reliably, and our goal is to close this gap while lifting both metrics.

Backbone	Method	Pass@1	Pass ¹	Pass@2	Pass ²	Δ_2	Pass@3	Pass ³	Δ_3
GLM-5.2 (high)	baseline	71.9	71.9	80.6	66.7	13.9	82.7	62.8	19.9
	TRACE	93.6 ($\uparrow 21.7$)	93.6 ($\uparrow 21.7$)	95.2 ($\uparrow 14.6$)	89.4 ($\uparrow 22.7$)	5.8 ($\downarrow 8.1$)	96.8 ($\uparrow 14.1$)	84.8 ($\uparrow 22.0$)	12.0 ($\downarrow 7.9$)
GPT-5.5 (medium)	baseline	75.6	75.6	83.3	67.7	15.6	87.7	59.9	27.8
	TRACE	97.8 ($\uparrow 22.2$)	97.8 ($\uparrow 22.2$)	98.1 ($\uparrow 14.8$)	96.2 ($\uparrow 28.5$)	1.9 ($\downarrow 13.7$)	98.5 ($\uparrow 10.8$)	94.5 ($\uparrow 34.6$)	4.0 ($\downarrow 23.8$)

Table 1: Consistency (Pass^k) and potential (Pass@k) on the full CAR-bench dataset (%), for $k \in \{1, 2, 3\}$. $\Delta_k = \text{Pass}@k - \text{Pass}^k$ measures the consistency gap (smaller is better). The best results are highlighted in **bold**. Each backbone is annotated with its reasoning effort.

Backbone	Method	Base	Hallu.	Disamb.
GLM-5.2 (high)	baseline	76.0	64.3	48.2
	TRACE	90.0	80.6	83.9
GPT-5.5 (medium)	baseline	67.0	73.5	39.3
	TRACE	95.0	93.9	94.6

Table 2: Pass³ broken down by task type on the full CAR-bench dataset (%): Base, Hallucination (Hallu.), and Disambiguation (Disamb.). Each backbone is annotated with its reasoning effort.

3.1 Experimental Setup

We use CAR-bench for both skill optimization and evaluation. The self-evolution loop first bootstraps the Skill Bank on the training split and then broadens its coverage using the test split, so that the resulting skills span a broader range of tasks. We report all numbers on the combined dataset of the training and test splits. To test the generality of the evolved Skill Bank across different underlying models, we run every experiment on two backbones: **GPT-5.5** with medium reasoning effort and **GLM-5.2** with high reasoning effort. The Skill Bank is evolved iteratively using trajectories collected from a GPT-5.5, and is then applied unchanged to both backbones at test time. For each backbone we compare two configurations:

- **baseline**: the model with its default prompt and native tool-calling, without any skill retrieval.
- **TRACE (Ours)**: the same model augmented with our self-evolved Skill Bank via the Actor’s per-turn retrieval.

Each task is run 3 times, and we report both metrics macro-averaged over the three task types.

3.2 Main Results

Tables 1 and 2 report all results on the full CAR-bench dataset. Table 1 gives the overall Pass^k and Pass@k for $k \in \{1, 2, 3\}$, and Table 2 breaks Pass³ down by the three task types. Since the two configurations share the same model, base prompt, and native tool-calling and differ only in whether the self-evolved Skill Bank is retrieved, the gap between the corresponding rows isolates the contribution of the skills derived from TRACE.

Overall Reliability and Gap Closure. The baselines expose precisely the gap CAR-bench targets: Pass^k decays with k while Pass@k rises, leaving a 19.9-point spread on GLM-5.2 (Pass³ 62.8% vs. Pass@3 82.7%) and 27.8 on GPT-5.5 (59.9% vs. 87.7%) between what a model *can* do and what it does on *every* trial. TRACE lifts reliability on

both backbones: Pass³ rises to 84.8% (+22.0) on GLM-5.2 and 94.5% (+34.6) on GPT-5.5, shrinking the Pass³-to-Pass@3 gap to 12.0 and 4.0 points. The gains thus favor reliability over single-trial capability: Pass³ improves by 22.0 points on GLM-5.2 and 34.6 points on GPT-5.5, compared with 21.7 and 22.2 points for Pass¹, respectively.

Skill Evolution and Cross-Backbone Transfer. As the baseline and TRACE configurations differ only in skill retrieval, contrasting the two rows provides a controlled comparison of the final Skill Bank’s effect: adding it is the only change to the pipeline, yet every cell of Table 1 improves substantially on both backbones (e.g. Pass³ by +22.0 and +34.6 points), so the gains stem from the evolved competencies rather than the model, prompt, or tooling. Because this bank was evolved solely on GPT-5.5 trajectories yet applied unchanged to GLM-5.2, its comparable GLM-5.2 gains further show the competencies transfer across backbones rather than overfitting their source model.

Performance across Task Types. Both baselines are highly uneven (Table 2), and *Disambiguation* is by far the weakest type on both backbones (48.2% on GLM-5.2 and 39.3% on GPT-5.5). Their relative strengths otherwise differ: GLM-5.2 is strongest on *Base* (76.0%), whereas GPT-5.5 is strongest on *Hallucination* (73.5%), underscoring the challenge of consistently resolving ambiguity. TRACE improves every type but most where the baseline is weakest: *Disambiguation* rises to 83.9% (+35.7) on GLM-5.2 and 94.6% (+55.3, its largest single gain) on GPT-5.5. This flattens the profile, narrowing the cross-type spread from 27.8 to 9.4 points on GLM-5.2 and 34.2 to 1.1 on GPT-5.5, so the evolved skills concentrate their effect on the hard, safety-critical behaviors that motivate the benchmark.

4 Conclusions and Limitations

We presented **TRACE**: a skill-based agent whose behavioral knowledge is a self-evolving Skill Bank that rewrites modular markdown competencies from clustered, runtime-faithful evaluation evidence under a strict de-hardcoding guide. At inference TRACE adds only lightweight skill retrieval on top of the base model, keeping its runtime overhead small, but this simplicity also bounds the method. Retrieval is performed by the LLM reading the resident skill descriptions, which suits the small bank used here but scales poorly as the bank grows. The Skill Bank also acts as a forward guide, with no feedback channel from execution back into skills at inference time, adding such online feedback to adjust or correct behavior mid-dialogue is a natural next step.

References

- [Barres *et al.*, 2025] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [Chen *et al.*, 2026] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026.
- [Guo *et al.*, 2025] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [Han *et al.*, 2025] Guangzeng Han, Weisi Liu, and Xiaolei Huang. Attributes as textual genes: Leveraging LLMs as genetic algorithm simulators for conditional synthetic data generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 19367–19389, 2025.
- [Hu *et al.*, 2025] Zican Hu, Wei Liu, Xiaoye Qu, Xiangyu Yue, Chunlin Chen, Zhi Wang, and Yu Cheng. Divide and conquer: Grounding LLMs as efficient decision-making agents via offline hierarchical reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 24570–24590, 2025.
- [Hu *et al.*, 2026] Zican Hu, Shilin Zhang, Yafu Li, Jianhao Yan, Xuyang Hu, Leyang Cui, Xiaoye Qu, Chunlin Chen, Yu Cheng, and Zhi Wang. Diversity-incentivized exploration for versatile reasoning. In *Proceedings of International Conference on Learning Representations*, 2026.
- [Jaech *et al.*, 2024] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [Jimenez *et al.*, 2024] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.
- [Kalai *et al.*, 2025] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, 2026.
- [Shinn *et al.*, 2023] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.
- [Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of International Conference on Learning Representations*, 2023.
- [Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [Wu *et al.*, 2026] Wenhao Wu, Zhentao Tang, Yafu Li, Shixiong Kai, Mingxuan Yuan, Zhenhong Sun, Chunlin Chen, and Zhi Wang. From conflict to consensus: Boosting medical reasoning via multi-round agentic RAG. In *Forty-third International Conference on Machine Learning*, 2026.
- [Yan *et al.*, 2025] Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance. In *Advances in Neural Information Processing Systems*, volume 38, pages 117157–117186, 2025.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.
- [Zhan *et al.*, 2026] Runzhe Zhan, Yafu Li, Zhi Wang, Xiaoye Qu, Dongrui Liu, Jing Shao, Derek F Wong, and Yu Cheng. ExGRPO: Learning to reason from experience. In *Proceedings of International Conference on Learning Representations*, 2026.
- [Zhang *et al.*, 2026] Haoran Zhang, Yafu Li, Zhi Wang, Zhilin Wang, Shunkai Zhang, Xiaoye Qu, and Yu Cheng. Characterizing, evaluating, and optimizing complex reasoning. In *Forty-third International Conference on Machine Learning*, 2026.